

VISUALISATION

Visualization is a way of representing. Data visualization is representing of data in a graphical or pictorial format for better understanding of data. Visualization gives a good idea of data and the trends in it.

In python, data visualization has multiple libraries like matplotlib, seaborn, plotly etc.

Let us dive deeper into matplotlib and seaborn.

MATPLOTLIB

Matplotlib, introduced in 2002 by John Hunter. It is for plotting 2D plots which is built on numpy. A plot of matplotlib contains:

1 Figure

2 Axes

3 Axis

4 Artists

Figure

Figure is the container in which plots are present. It can have a single plot or multiple plots.

Axes

Axes are the plots, and they are artist attached to the figure. A plot can have 2 or 3 axes. `set_xlabel()`, `set_ylabel()` are the functions used to set the labels of x and y respectively.

Axis

It is responsible for generating ticks or the limits on the axes.

Artists

The whatever content visible in a figure are the artists.

PYLOT

Pyplot is a sub library of matplotlib where all the utilities lie under. It has different types of plots including bar graphs, scatter plots, pie charts, histograms, area charts.

We import pyplot from matplotlib as following:

```
1 import matplotlib.pyplot as plt
```

We also import numpy as a support for the matplotlib :

```
import numpy as np
```

Plots in Matplotlib :

LINE CHART

Line plots are the basic charts where dot consecutive points are connected by a continuous line.

The default plot() is used for line charts.

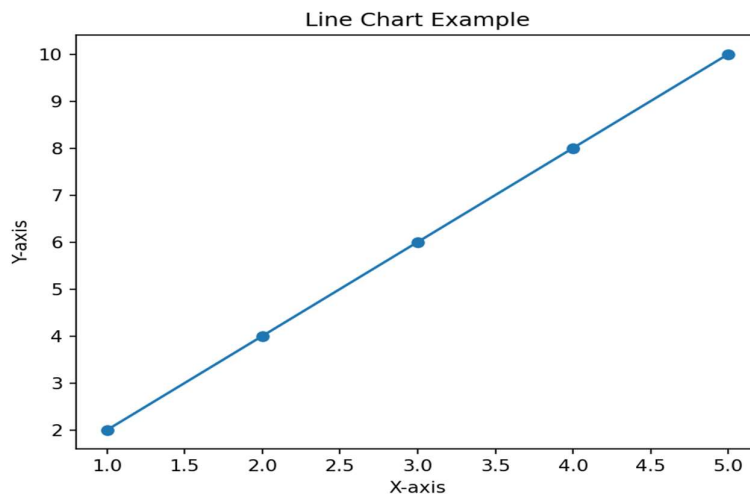
Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

plt.plot(x, y, marker='o')
plt.title('Line Chart Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

Output :



Description:

np.array() is defining array of the respective values of x and y.

plot() for the plotting the line chart.

title() for defining the title, ylabel() and xlabel () for labelling y and x axis respectively.

show() displays the graph.

BAR GRAPH

Rectangular bars are used in bar chart as the height represent the frequency of a particular element. `bar()` is used for plotting bar graphs, it has parameters like x, y, width, color.

`bar(x, y, color, width)`

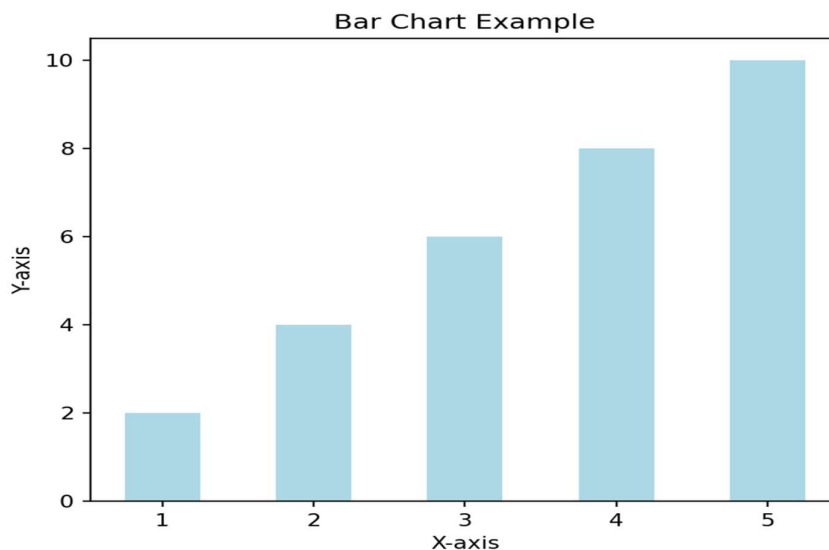
Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

fig = plt.figure(figsize=(4, 4))
plt.bar(x, y, width = 0.5, color = 'lightblue')
plt.title('Bar Chart Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

Output :



Description:

`np.array()` shows the array with specific values.

`bar()` is used for plotting the bar chart with parameters. width is used to define the width of the graph and color is used to set the colour of the plot.

`plt.xlabel()` and `plt.ylabel()` are used for x-axis and y-axis labels respectively.

`show()` displays the graph.

HISTOGRAM

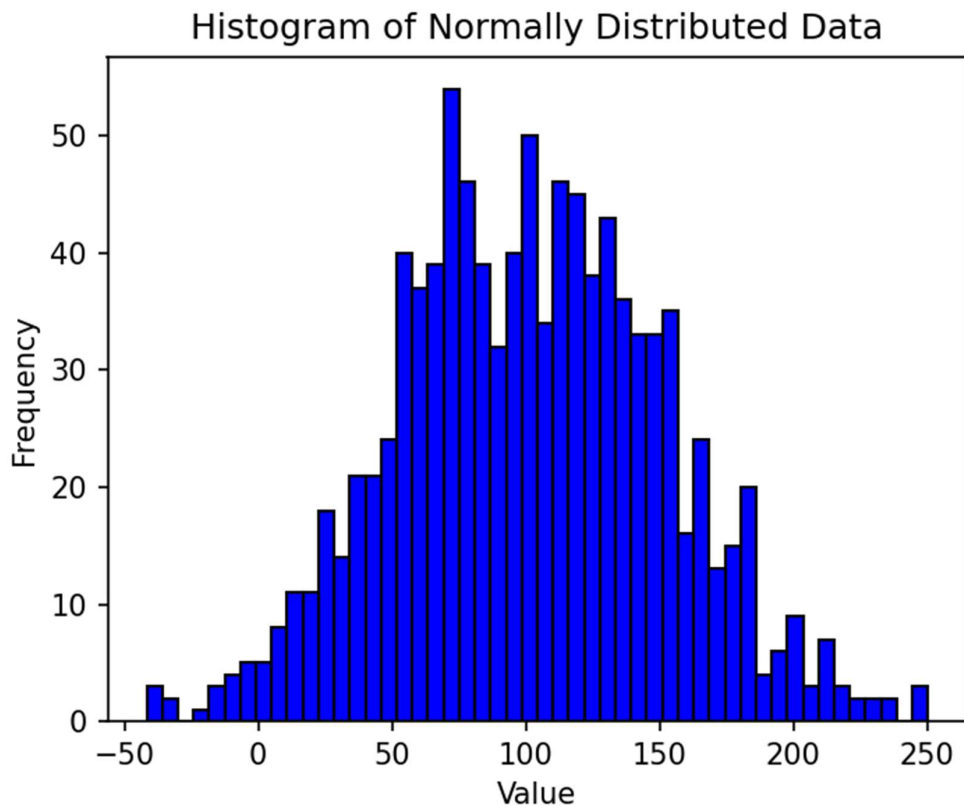
Histogram is a type of bar graph where the graph is represented in groups. hist() is used for plotting histogram with parameters x, bins, color, edgecolor.

Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(100, 50, 1000)
fig = plt.figure(figsize=(3, 3))
plt.hist(x, bins=50, color='blue', edgecolor='black')
plt.title('Histogram of Normally Distributed Data')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

Output :



Description:

hist() have parameters x which is a random generated array around one hundred with standard deviation 50 of 100 values.

bins indicate the number of sections on x axis.

color indicates the color of rectangular bars.

edge color that divides the rectangular bars.

SCATTER PLOT

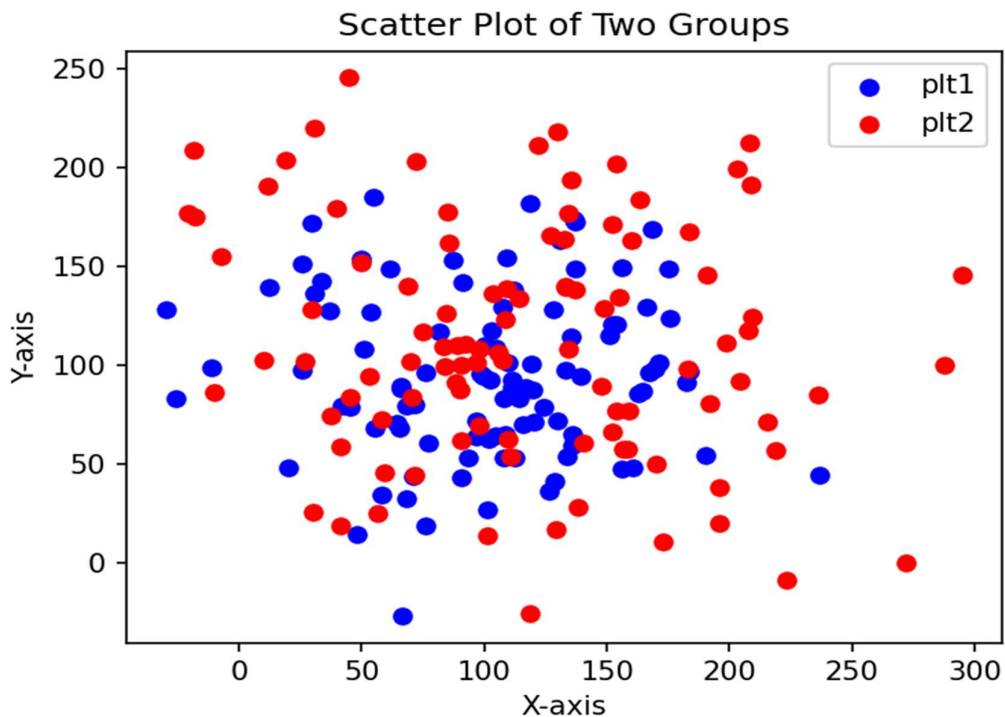
Scatter plot uses dots to depict the relationship between the data. `scatter()` is used for plotting scatter plot and scatter plot can be done for multiple datasets at the same time by differentiating it with colors. Legends help to achieve this difference.

Code snippet:

```
import matplotlib.pyplot as plt
import numpy as np

x1 = np.random.normal(100, 50, 100)
y1 = np.random.normal(100, 50, 100)
x2 = np.random.normal(120, 60, 100)
y2 = np.random.normal(120, 60, 100)
fig = plt.figure(figsize=(3, 3))
plt.scatter(x1, y1, color='blue', label='Group 1')
plt.scatter(x2, y2, color='red', label='Group 2')
plt.title('Scatter Plot of Two Groups')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend(['plt1', "plt2"])
plt.show()
```

Output :



Description:

`x1, y1` belong to one dataset and `x2, y2` belong to another dataset.

Here the `scatter()` is used for the scatter plot and the parameters `x`, `y` and `c` which represents color.

Legend adds the label which helps to differentiate the plots.

PIECHART

Pie charts are used to plot data of same kind which means the same series of data where the different elements are divide based on their percentage.

Code snippet:

```
from matplotlib import pyplot as plt
import numpy as np
time = [1, 2, 3, 4, 5]
colors = ['red', 'blue', 'green', 'yellow', 'cyan']
work = ["Sleeping", "Eating", "Working", "Playing", "Others"]
fig = plt.figure(figsize = (4,4))
plt.pie(time, labels=work, wedgeprops={"edgecolor":"black","linewidth":1}, colors=colors)
plt.title("My Daily Activities")
plt.show()
```

Output :



Description:

Here in the code, time is the x variable and labels are defined using labels keyword.

wedgeprops attribute is used to define the linewidth and the color to separate the elements in the pie chart.

color array contains the colors to each element.

AREA CHART

In area chart, the area under the line to x axis is filled or shaded. It can be done by using `fill_between()` function or `stackplot()` function.

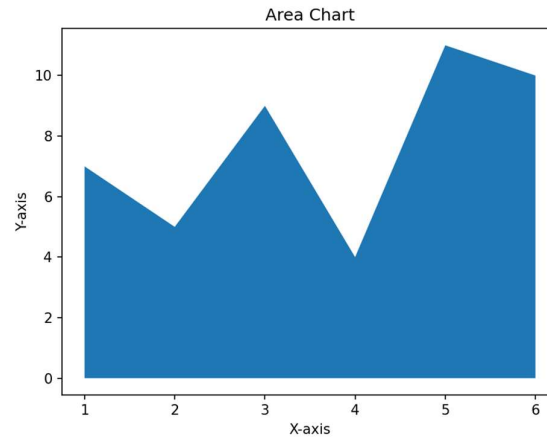
Code snippet:

Using `fill_between()` function :

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5, 6])
y = np.array([7, 5, 9, 4, 11, 10])

plt.fill_between(x, y)
plt.title("Area Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

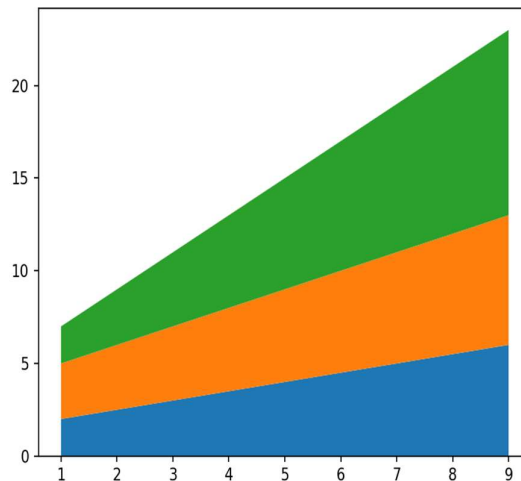


Using `stackplot()` function :

```
import numpy as np
import matplotlib.pyplot as plt

x = [1, 3, 5, 7, 9]
y1 = [2, 3, 4, 5, 6]
y2 = [3, 4, 5, 6, 7]
y3 = [2, 4, 6, 8, 10]

plt.stackplot(x, y1, y2, y3)
plt.show()
```



Description:

`fill_between()` function have parameters of x and y.

`stackplot()` function parameters are x, y1, y2, y3 where y1, y2, y3 are the different groups.

Legends are added by mentioning the labels attribute in `stackplot()` function and giving the position to the legend using `plt.legend()` function with attribute loc.

PANDAS

Pandas, introduced in 2008 by Wes McKinney. It is for data manipulation and analysis which is built on NumPy. The core components of pandas include:

- DataFrame
- Series
- Index
- GroupBy

DataFrame

DataFrame is the primary two-dimensional, tabular data structure with labeled rows and columns. It can be created from dictionaries, lists, or external files like CSV.

Series

Series is a one-dimensional labeled array capable of holding any data type. It is like a single column of a DataFrame, and functions like `mean()`, `sum()` can be applied directly.

Index

Index holds the row labels of a DataFrame or Series. It provides alignment, slicing, and fast lookup. Methods like `set_index()`, `reset_index()` are used to manipulate it.

GroupBy

GroupBy is used for split-apply-combine operations. It allows grouping data by one or more columns and applying aggregation functions like `sum()`, `mean()`, or custom functions.

We import pandas and numpy as following :

```
In [1]: import numpy as np
In [2]: import pandas as pd
```

Below are some graphs using pandas :

SCATTER PLOT

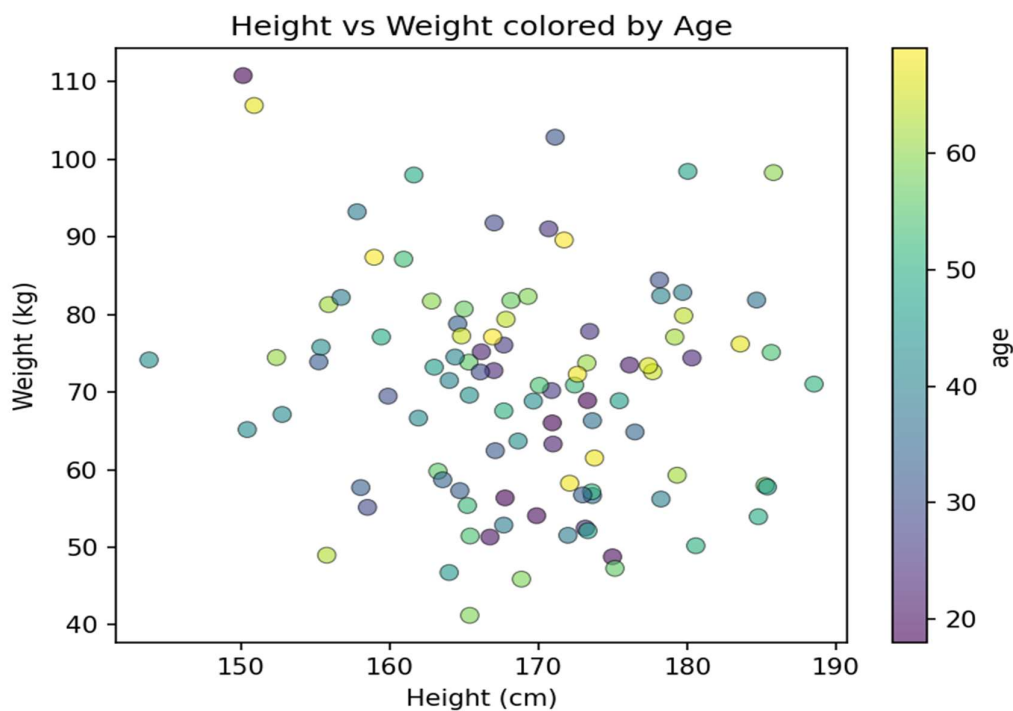
Scatter plot uses dots to depict the relationship between the data. `scatter()` is used for plotting scatter plot and scatter plot can be done for multiple datasets at the same time by differentiating it with colors. Legends help to achieve this difference.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create random data
np.random.seed(42)
df = pd.DataFrame({
    'height': np.random.normal(170, 10, 100),
    'weight': np.random.normal(70, 15, 100),
    'age': np.random.randint(18, 70, 100)
})

# Customized scatter plot
df.plot.scatter(
    x='height',
    y='weight',
    c='age',          # color points by 'age' column
    colormap='viridis', # color map for 'c'
    s=50,            # marker size
    alpha=0.6,       # transparency
    title='Height vs Weight colored by Age',
    xlabel='Height (cm)',
    ylabel='Weight (kg)',
    edgecolors='black',
    linewidth=0.5
)
plt.show()
```

Output :



Description :

- Uses `df.plot.scatter(x='height', y='weight')` to plot two numeric columns from a DataFrame.
- Colors points using the `c='age'` parameter and applies a colormap (`colormap='viridis'`) to represent values of another column.
- Adjusts marker size (`s=50`), transparency (`alpha=0.6`), edges (`edgecolors='black'`), and adds title and axis labels.
- The plot is displayed with `plt.show()` and requires Matplotlib to be installed.

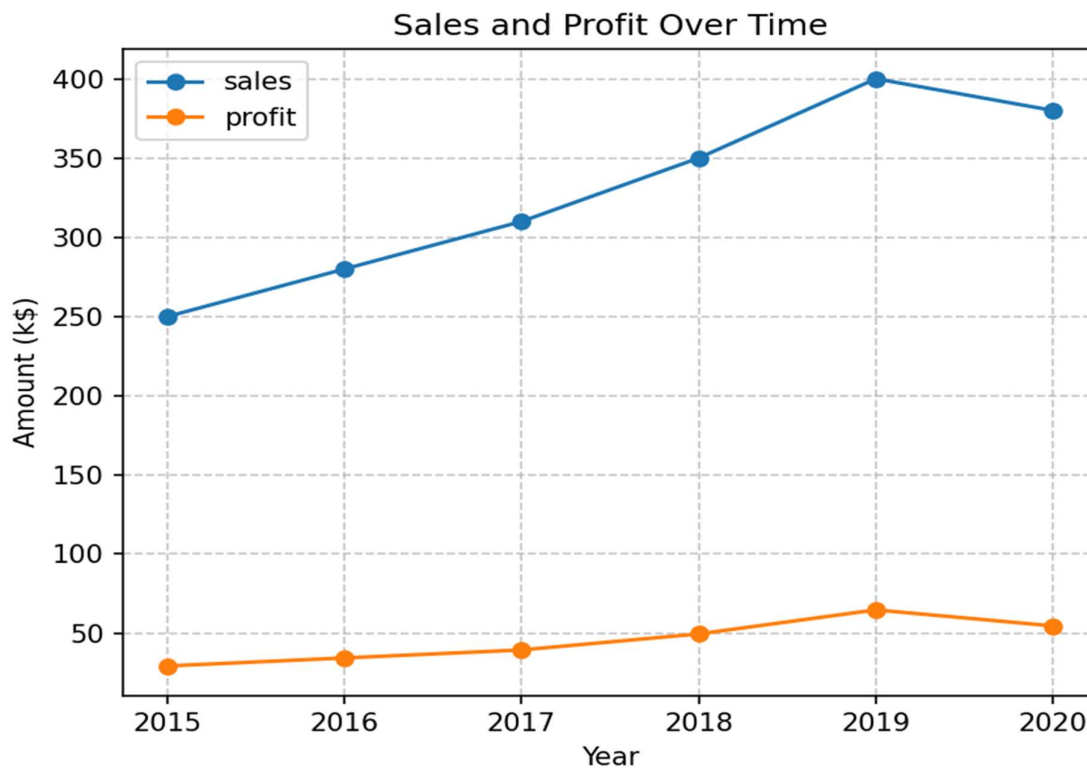
LINE PLOT

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample time-series data
df = pd.DataFrame({
    'year': [2015, 2016, 2017, 2018, 2019, 2020],
    'sales': [250, 280, 310, 350, 400, 380],
    'profit': [30, 35, 40, 50, 65, 55]
})

# Line chart with two lines
df.plot.line(x='year', y=['sales', 'profit'], marker='o', title='Sales and Profit Over Time')
plt.xlabel('Year')
plt.ylabel('Amount (k$)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Output :



Description :

- Uses `df.plot.line(x='year', y='sales')` to create a line chart from two DataFrame columns.
- Pass a list to `y=` (e.g., `['sales','profit']`) to plot several columns on the same axes.
- `marker='o'` adds circular markers at data points; `linestyle='--'` or `linewidth` can be adjusted.
- Adds grid lines, axis labels, and a title for clarity. Display with `plt.show()`.

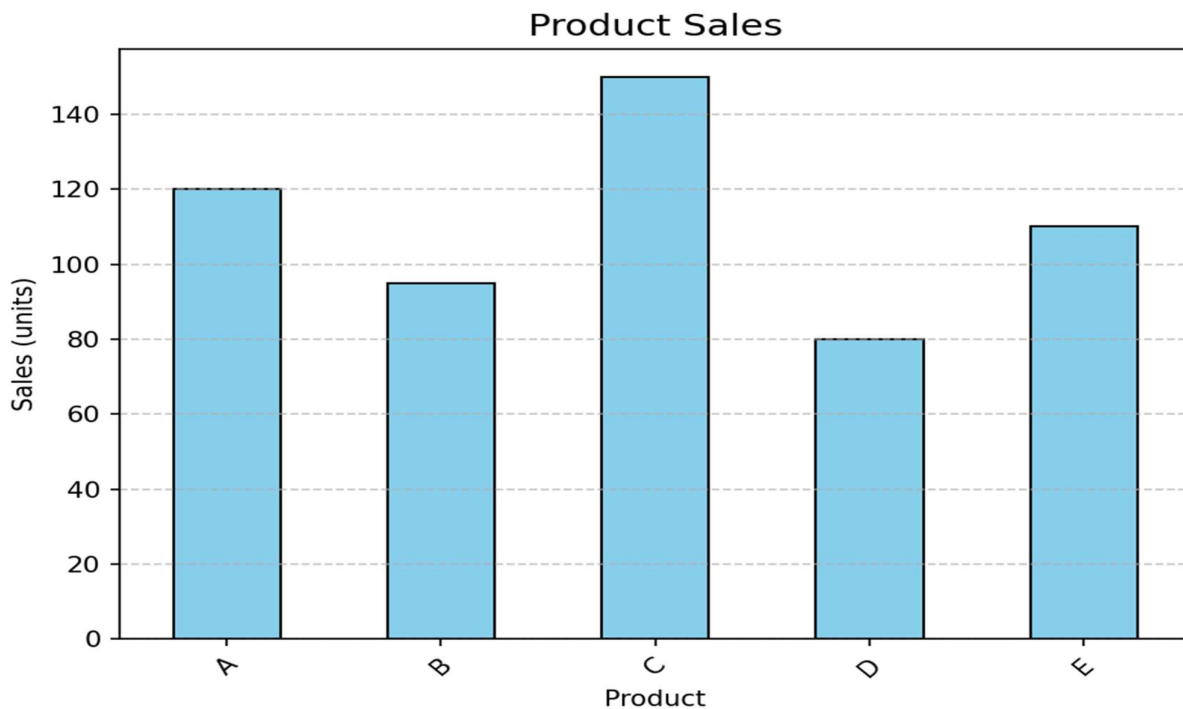
BAR PLOT

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample data: product sales
df = pd.DataFrame({
    'product': ['A', 'B', 'C', 'D', 'E'],
    'sales': [120, 95, 150, 80, 110],
    'profit': [30, 25, 40, 18, 35]
})

# Bar plot
df.plot.bar(x='product', y='sales', color='skyblue', edgecolor='black', legend=False)
plt.title('Product Sales', fontsize=14)
plt.xlabel('Product')
plt.ylabel('Sales (units)')
plt.xticks(rotation=45) # Rotate x labels if needed
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Output :



Description :

- Use `df.plot.bar(x='product', y='sales')` to create vertical bars from DataFrame columns.
- Pass a list to `y=` (e.g., `['sales', 'profit']`) to display multiple bars per category side-by-side.
- Change bar colors (`color`), add borders (`edgecolor`), rotate x-axis labels (`rotation`), and add grid lines.
- Use `df.plot.barh()` for horizontal orientation. Display with `plt.show()` and save with `plt.savefig()`.

HISTOGRAM

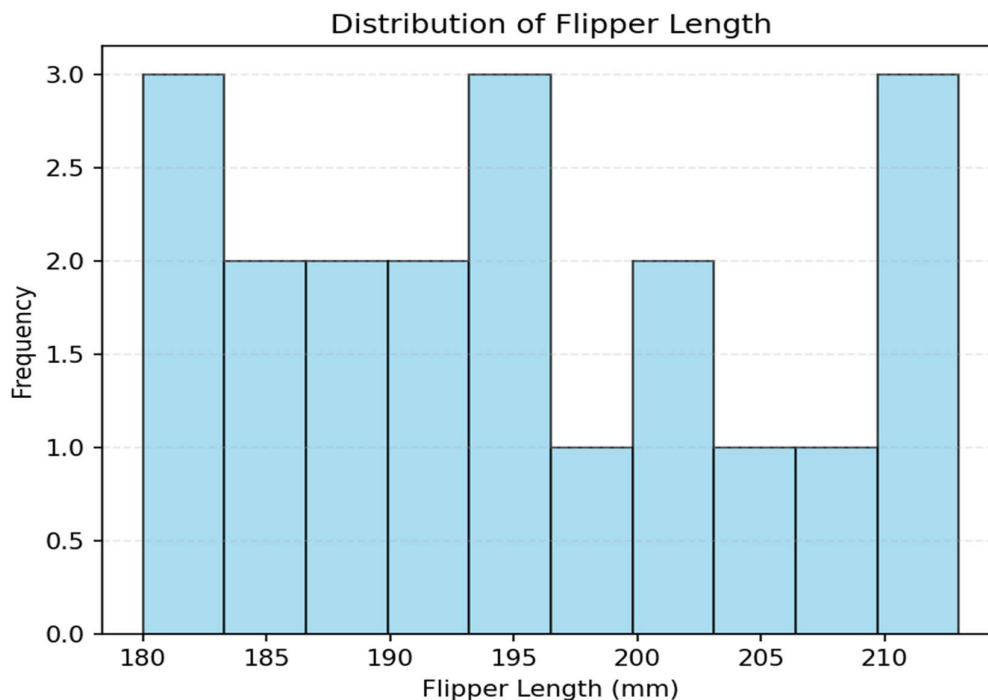
```
import pandas as pd
import matplotlib.pyplot as plt

# Sample DataFrame with a numeric column
df = pd.DataFrame({
    'flipper_length_mm': [181, 195, 210, 185, 198, 205, 182, 212, 196, 188,
                        190, 193, 200, 202, 186, 189, 194, 208, 213, 180]
})

# Histogram using pandas
df['flipper_length_mm'].plot.hist(
    bins=10,          # number of bins
    edgecolor='black', # bar borders
    color='skyblue',
    alpha=0.7,
    title='Distribution of Flipper Length'
)

plt.xlabel('Flipper Length (mm)')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```

Output :



Description :

- Use `df['column'].plot.hist(bins=10)` to show frequency distribution of a numeric variable.
- Adjust number of bins, colors, transparency (`alpha`), and add bar borders.
- Pass multiple columns to `df.plot.hist()` for overlapping histograms (good for comparison).

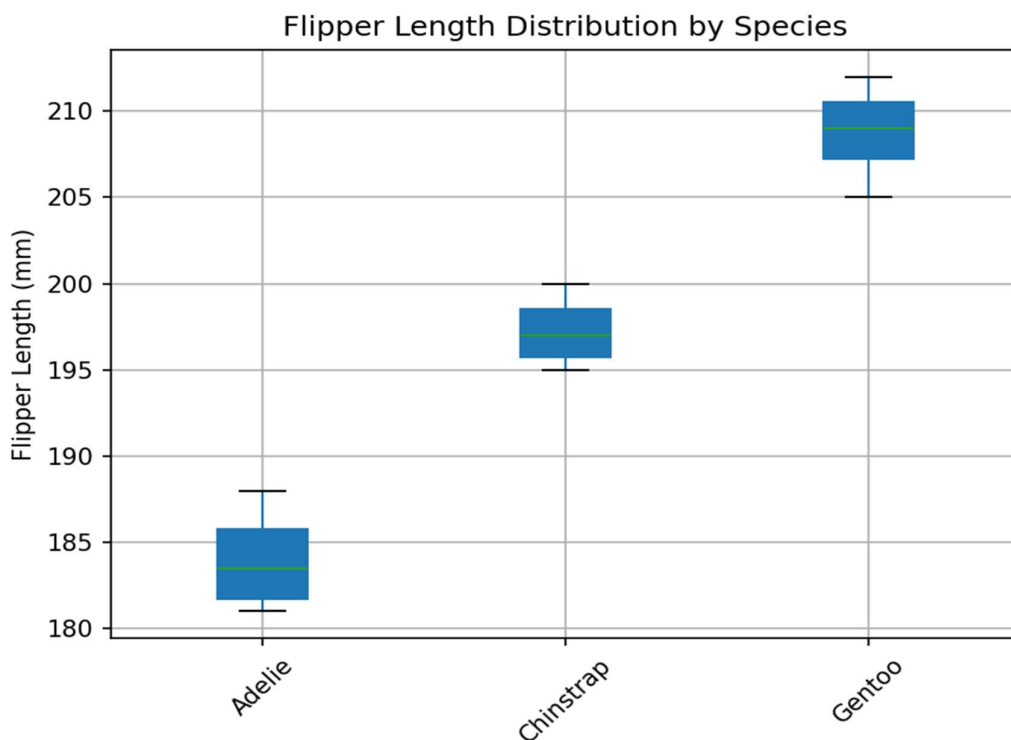
BOX PLOT

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample DataFrame with numeric columns
df = pd.DataFrame({
    'species': ['Adelie']*4 + ['Chinstrap']*4 + ['Gentoo']*4,
    'flipper_length': [181, 185, 182, 188, 195, 198, 196, 200, 210, 212, 205, 208]
})

# Box plot grouped by species
df.boxplot(column='flipper_length', by='species', grid=True, patch_artist=True)
plt.title('Flipper Length Distribution by Species')
plt.suptitle('') # Remove default "Boxplot grouped by..." title
plt.xlabel('Species')
plt.ylabel('Flipper Length (mm)')
plt.xticks(rotation=45)
plt.show()
```

Output :



Description :

1. Use `df.boxplot(column='numeric_col', by='cat_col')` to compare distributions across categories.
2. With `.plot.box()`, first pivot the data so each category becomes a separate column.
3. `patch_artist=True` fills boxes; `color dict` controls box, whisker, and median colors.
4. Box plots automatically show outliers as circles beyond whiskers ($1.5 \times \text{IQR}$ rule).

DENSITY PLOT

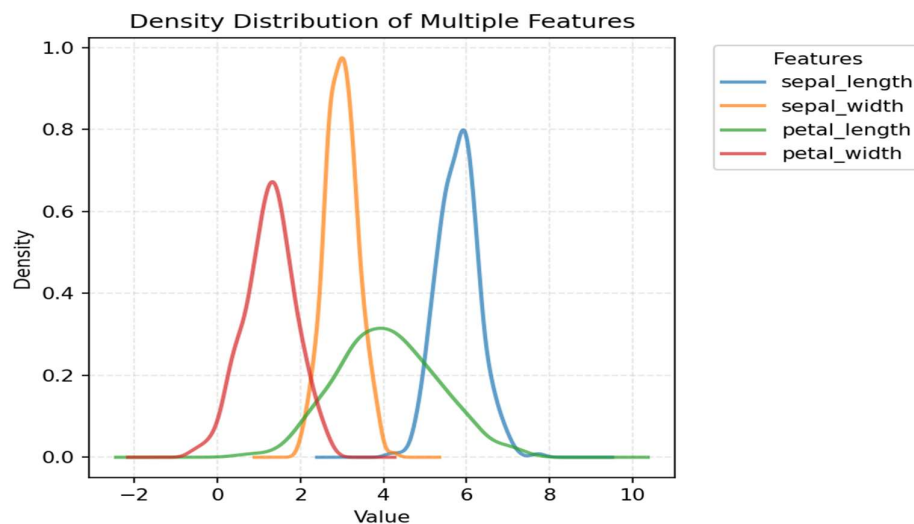
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Sample DataFrame with multiple numeric features (columns)
np.random.seed(42)
df = pd.DataFrame({
    'sepal_length': np.random.normal(5.8, 0.5, 300),
    'sepal_width': np.random.normal(3.0, 0.4, 300),
    'petal_length': np.random.normal(4.0, 1.2, 300),
    'petal_width': np.random.normal(1.2, 0.6, 300)
})

# Density plot for all features
df.plot.density(
    linewidth=2,
    alpha=0.7,
    title='Density Distribution of Multiple Features'
)

# Customization using matplotlib
plt.xlabel('Value')
plt.ylabel('Density')
plt.grid(True, linestyle='--', alpha=0.3)
plt.legend(title='Features', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

Output :



Description :

- Uses a DataFrame with several numeric columns (e.g., `sepal_length`, `sepal_width`) and plots a separate KDE curve for each column on the same axes.
- Calls `df.plot.density()` directly on the DataFrame, which automatically generates density plots for all numeric columns without loops.
- Pandas' density/KDE method depends on `scipy.stats.gaussian_kde`; the code includes an optional manual KDE implementation using only NumPy/Matplotlib as a fallback.
- Adjusts line width, transparency (`alpha`), title, grid, and legend placement to improve readability and comparison of distributions.

REG PLOT

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

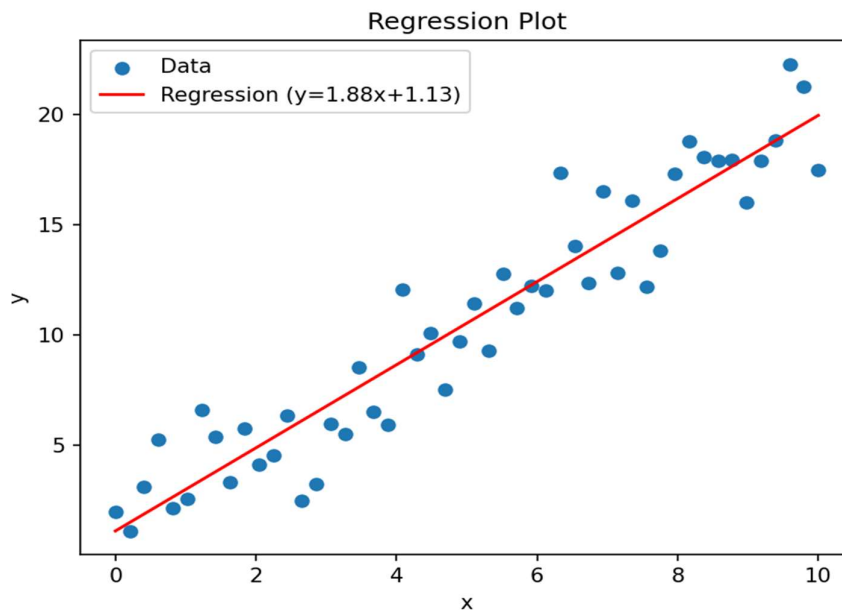
# Generate random data
np.random.seed(42)
x = np.linspace(0, 10, 50)
y = 2 * x + 1 + np.random.normal(0, 2, size=len(x))

# Create DataFrame
df = pd.DataFrame({'x': x, 'y': y})

# Fit regression line
slope, intercept = np.polyfit(df['x'], df['y'], 1)
reg_line = slope * df['x'] + intercept

# Plot
plt.scatter(df['x'], df['y'], label='Data')
plt.plot(df['x'], reg_line, color='red', label=f'Regression (y={slope:.2f}x+{intercept:.2f})')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Regression Plot')
plt.legend()
plt.show()
```

Output :



Description :

- **Generates synthetic data** – Creates 50 random (x, y) points with a linear trend ($y = 2x + 1$) plus Gaussian noise using NumPy, then stores them in a pandas DataFrame.
- **Fits a regression line** – Uses `numpy.polyfit` to compute the slope and intercept of the best-fit line, then calculates the corresponding regression values.
- **Plots the result** – Displays a scatter plot of the original data points alongside the fitted red regression line, complete with axis labels, a title, and a legend showing the equation.

HEATMAP

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Generate random DataFrame with numeric and categorical columns
np.random.seed(42)
n = 100
df = pd.DataFrame({
    'length': np.random.normal(40, 5, n),
    'depth': np.random.normal(15, 2, n),
    'weight': np.random.normal(4000, 500, n),
    'species': np.random.choice(['Adelie', 'Chinstrap', 'Gentoo'], n),
    'island': np.random.choice(['Torgersen', 'Biscoe', 'Dream'], n),
    'sex': np.random.choice(['Male', 'Female'], n)
})

# Encode categorical and plot correlation heatmap
df_encoded = pd.get_dummies(df, drop_first=True)
plt.figure(figsize=(6, 6))
sns.heatmap(df_encoded.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap of Random Data")
plt.tight_layout()
plt.show()
```

Output :



Description :

- Generates a synthetic DataFrame with 100 rows of numeric features (length, depth, weight) and categorical features (species, island, sex) using NumPy.
- Encodes categorical variables into dummy/indicator columns with `pd.get_dummies(drop_first=True)`, then computes the correlation matrix of all numeric columns.
- Displays a correlation heatmap using `seaborn.heatmap` with annotations and the `coolwarm` colormap, showing pairwise relationships in the random data.

HEXBIN PLOT

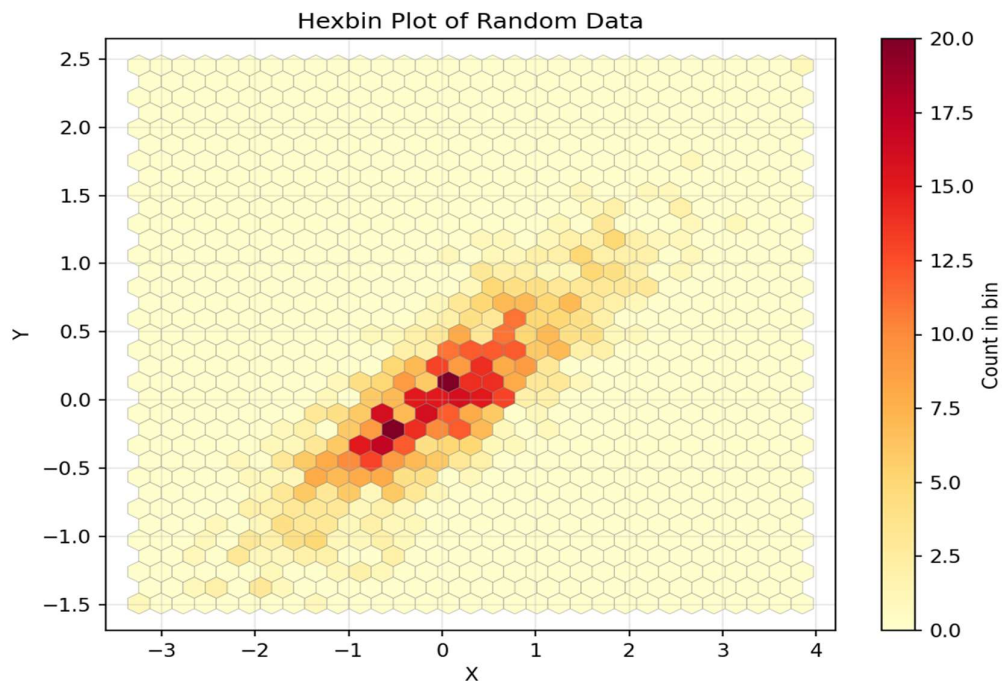
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Random data
np.random.seed(42)
x = np.random.randn(1000)
y = x * 0.5 + np.random.randn(1000) * 0.3

df = pd.DataFrame({'x': x, 'y': y})

plt.figure(figsize=(8, 6))
hb = plt.hexbin(df['x'], df['y'], gridsize=30, cmap='YlOrRd', edgecolors='gray', linewidths=0.2)
plt.colorbar(hb, label='Count in bin')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Hexbin Plot of Random Data')
plt.grid(alpha=0.3)
plt.show()
```

Output :



Description :

- Generates 1000 random (x, y) points with a weak correlation and stores them in a pandas DataFrame.
- Creates a hexbin plot using `plt.hexbin()` with a soft YlOrRd colormap, gray hexagon borders, and 30 bins across for clearer density visualization.
- Adds a colorbar, labels, title, and a light grid to enhance readability and make the graph eye-friendly.

COMPARISON OF MATPLOTLIB AND SEABORN

Matplotlib:

- **Core Library:** Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a low-level interface for creating plots with fine-grained control over every aspect of the figure.
- **Flexibility:** Matplotlib allows users to create any type of plot imaginable, from basic line plots and scatter plots to complex 3D plots and geographical maps.
- **Mature:** Matplotlib has been around for a long time and is a well-established library in the Python ecosystem. Many other libraries, including Seaborn, are built on top of Matplotlib.
- **Customization:** Matplotlib offers extensive customization options, allowing users to customize every aspect of a plot, including colors, line styles, markers, fonts, annotations, and more.
- **Integration:** Matplotlib can be easily integrated with other libraries and frameworks, such as NumPy, Pandas, SciPy, which makes it a versatile choice for data visualization in various contexts.

Advantages of Matplotlib:

- High degree of customization and control over plots.
- Wide range of plot types and styles.
- Strong community support and extensive documentation.
- Well-suited for creating publication-quality figures and graphics.

Pandas:

- **Built-in Plotting:** Pandas provides a lightweight `.plot()` method for Series and DataFrames, which is a wrapper around Matplotlib.
- **Convenience:** Creates basic plots (line, bar, hist, box, scatter, etc.) directly from data structures with minimal code.
- **Default Aesthetics:** Uses Matplotlib's defaults but offers simple shortcuts for common tasks (e.g., `df.plot(kind='bar')` or `df.hist()`).
- **Integration:** Tightly coupled with Pandas data structures – no need to extract columns manually; missing data is handled automatically.

Advantages of Pandas:

- Extremely concise syntax – one line to plot a whole DataFrame.
- Automatically uses DataFrame indices and column names for labels.
- Great for exploratory data analysis (EDA) and quick checks.
- Lighter learning curve for beginners who already use Pandas.

Overall, the choice between Matplotlib and Pandas depends on your goal. Pandas is excellent for quick, simple plots directly from data frames with minimal effort. Matplotlib is necessary when you need detailed customization, multi-panel figures, or specialized plot types that Pandas' `.plot()` does not support. In practice, many users start with Pandas' quick plots and then switch to Matplotlib for fine-tuning and final presentation.